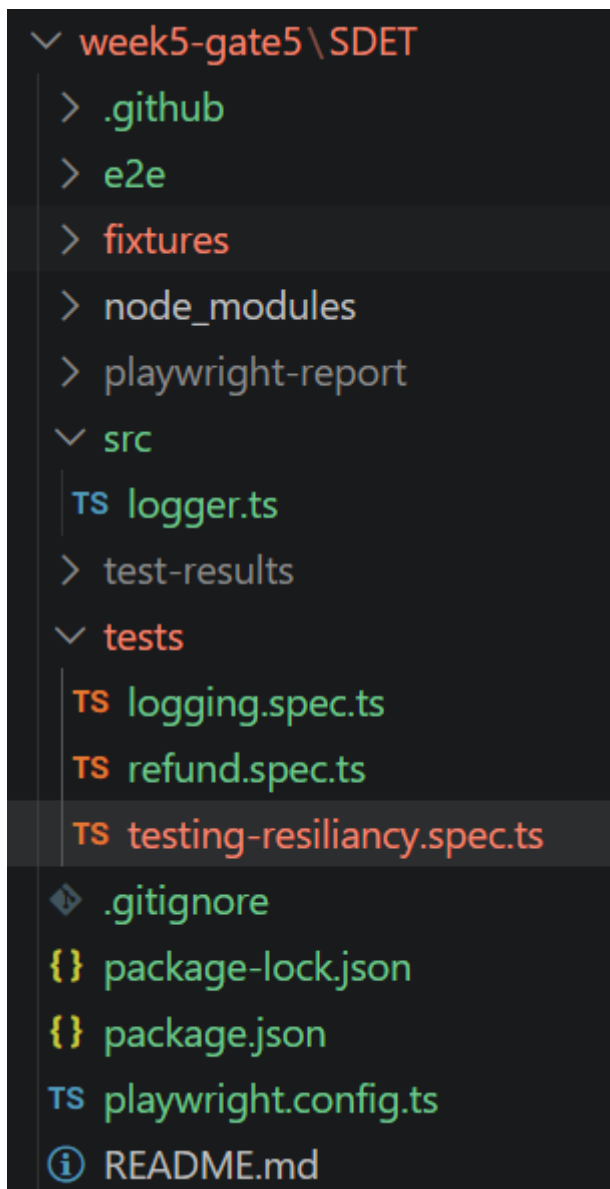


Report For Week5 Gate5 Assessment

Tools Used

- Playwright
- TypeScript
- Winston Logger
- Playwright Fixtures

Folder Structure



Test1 : Diagnostics and Logging

Structured Logging

A custom logger is created using Winston.

The following events are logged:

- Checkout started
- Cart item added
- Cart details loaded
- Order placed
- Test completed

Cart Response Evidence

After loading the cart, the API response is stored as evidence.

```
{  
  "items":[...],  
  "total":4499  
}
```

This file is attached as:

cart-response.json

Diagnosis Evidence

After the order is successfully created, a diagnosis message is prepared.

Checkout completed successfully

Order Number : ORD-1001

Status : Confirmed

Total : 4598

This is attached as:

diagnosis.txt

Log Evidence

Execution steps are stored in a log list.

Checkout journey started

Cart item added

Cart snapshot loaded

Order placed

These logs are attached as:

logs.txt

Evidence Generated

After successful execution, the following evidence files are generated:

- cart-response.json
- diagnosis.txt
- logs.txt
- Playwright Trace
- Screenshot (on failure)
- Video (if enabled)

These files help in analyzing the execution and identifying issues quickly

Failure

The status code of the add item should be 201 to fail the test case I given 200 so it is failing.

```
productid:101
});
evidence.logs?.push("Cart item added");
//the status code should be 201 i given 200 so it failed after i changed to 201 then it is passing
expect(addItem.status()).toBe(200);

const cart=await request.get(`${postApiUrl}/api/cart`,{headers});
const cartBody=await cart.json();
evidence.cartResponse = cartBody;
log.debug("cart snapshot loaded",{

S 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
Users\304556\Desktop\testing\week5-gate5\SDET\tests> npx playwright test
ceived: 201

37 |         });
38 |         evidence.logs?.push("Cart item added");
39 |         expect(addItem.status()).toBe(200);
40 |
41 |         const cart=await request.get(`${postApiUrl}/api/cart`,{headers});
42 |         const cartBody=await cart.json();
at C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests\logging.spec.ts:39:34
```

Then to know the error I ran in trace mode by giving command **npx playwright test --trace on**.

> ✓ Before Hooks990ms

> ✓ POST "/api/cart/items" — logging.spec.ts:2254ms

> ✗ Expect "toBe" — logging.spec.ts:403ms

> ✓ After Hooks75ms

> ✓ Worker Cleanup534ms

Attachments

✓ cart-response.json

{
 "message": "No cart captured"
}

✓ diagnosis.txt

No diagnosis captured

✓ logs.txt

Checkout journey started
Cart item added

[logs.ndjson](#)
[error-context](#)

> stdout

I found the error and I changed status code to 201 so it is passing.

> ✓ Expect "toBeGreaterThan" — logging.spec.ts:541ms

> ✓ POST "/api/orders" — logging.spec.ts:575ms

> ✓ Expect "toBe" — logging.spec.ts:790ms

> ✓ Expect "toMatch" — logging.spec.ts:801ms

> ✓ Expect "toBe" — logging.spec.ts:810ms

> ✓ Expect "toBe" — logging.spec.ts:821ms

> ✓ After Hooks26ms

Attachments

> cart-response.json

> diagnosis.txt

✓ logs.txt

Checkout journey started
Cart item added
Cart snapshot loaded
Order placed

> stdout

Test2: Testing Resiliency

Test 1 – Rollback when Sync Fails

Description

This test verifies that if the application fails to synchronize a sale with the server, the optimistic sale is rolled back automatically.

Steps Performed

1. Opened the application.
2. Intercepted the **/api/sales** request.
3. Forced the API request to fail.
4. Clicked the **Queue Sale** button.
5. Verified that the queued sale was removed.
6. Verified that the application displayed the rollback message.

Expected Result

- Sale synchronization fails.
- Optimistic sale is removed.
- Outbox count becomes **0**.
- User receives a rollback message.

Test 2 – Retry with Same Idempotency Key

Description

This test verifies that the same queued sale is retried using the same Idempotency-Key and does not create duplicate sales.

Steps Performed

1. Opened the POS application.
2. Intercepted the Sales API.
3. Failed the first request.
4. Allowed the second request to succeed.
5. Verified the Idempotency-Key.

6. Verified that only one sale was synchronized.

Expected Result

- First request fails.
- Second request succeeds.
- Same Idempotency-Key is reused.
- Only one successful sale is created.

Test 3 – Offline Banner Detection

Description

This test verifies that the application correctly detects offline mode.

Steps Performed

1. Opened the POS page.
2. Verified Online status.
3. Switched the browser to Offline mode.
4. Verified Offline banner.
5. Checked the navigator.onLine property.

Expected Result

- Offline banner should appear.
- navigator.onLine should return **false**.

Test 4 – Queue Sale while Offline

Description

This test verifies that sales are stored in the outbox while the application is offline.

Steps Performed

1. Opened the POS page.
2. Switched to Offline mode.
3. Clicked Queue Sale.
4. Verified pending status.

5. Verified Outbox count.

Expected Result

- Sale should be stored in the outbox.
- Pending status should be displayed.
- Outbox count should be **1**.

Test 5 – Synchronize Sale after Reconnection

Description

This test verifies that queued sales are synchronized automatically after the application reconnects.

Steps Performed

1. Opened the POS page.
2. Switched Offline.
3. Queued one sale.
4. Switched Online.
5. Verified that the queued sale synchronized successfully.

Expected Result

- Outbox count changes from **1** to **0**.
- Sale is synchronized.
- API is called only once.

Test 6 – Evidence and Triage

Description

This test verifies that evidence is generated for debugging purposes.

Steps Performed

1. Started the POS application.
2. Switched Offline.
3. Queued a sale.

4. Switched Online.
5. Verified synchronization.
6. Attached evidence files.

Evidence Generated

- logs.ndjson
- diagnosis.txt
- cart-response.ndjson
- offline-sync-journey.txt
- Screenshot (Failure)
- Trace file (Failure)

Failure:

The Application should be offline in the test detects and displays offline banner

I changed to online so it is failing.

detects and displays offline banner

testing-resiliancy.spec.ts:122

5.8s

✖ Run

Errors

```
Error: expect(locator).toContainText(expected) failed
Locator: getByTestId('network-banner')
Expected pattern: /online/i
Received string: "Offline - sales will be queued"
Timeout: 5000ms

Call log:
- Expect "toContainText" with timeout 5000ms
  - waiting for getByTestId('network-banner')
    14 x locator resolved to <div role="status" data-online="false" data-testid="network-banner" class="network-banner o
      - unexpected value "Offline - sales will be queued"

127 |         evidence.logs?.push("Browser offline");
128 |         //This is wrong because the application is actually Offline i change to online so it will fail
> 129 |         await expect(page.getByTestId("network-banner")).toContainText(/online/i);
    |                                     ^
130 |         const online = await page.evaluate(() => navigator.onLine);
131 |         expect(online).toBe(false);
132 |         await context.setOffline(false);
    |         at C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests\testing-resiliancy.spec.ts:129:54
```

After the Trcaing I got to know that and changed to the offline so it is passing.

Filter steps

> ✓ Before Hooks	204ms
> ✓ Navigate to "/pos" — testing-resiliancy.spec.ts:8	465ms
> ✓ Expect "toBeVisible" getByRole('heading', { name: 'Resilient POS' }) — testing-resiliancy.spec.ts:9	134ms
> ✓ Expect "toContainText" getByTestId('network-banner') — testing-resiliancy.spec.ts:125	8ms
> ✓ Set offline mode — testing-resiliancy.spec.ts:126	2ms
> ✓ Expect "toContainText" getByTestId('network-banner') — testing-resiliancy.spec.ts:130	7ms
> ✓ Evaluate — testing-resiliancy.spec.ts:131	6ms
> ✓ Expect "toBe" — testing-resiliancy.spec.ts:132	0ms
> ✓ Set offline mode — testing-resiliancy.spec.ts:133	3ms
> ✓ After Hooks	34ms

Attachments

- > cart-response.json
- > diagnosis.txt
- > logs.txt

Test3 : Refund Tests

Test 1 – Verify Complete Refund

Description

This test verifies that when all purchased items are refunded, the order status changes to **REFUNDED** and the refundable balance becomes **0**.

Steps Performed

1. Created a refund test order.
2. Sent a refund request for all three items.
3. Retrieved the updated order details.
4. Verified the order status.
5. Verified the refundable balance.

Expected Result

- Refund request should return **201 Created**.
- Order status should become **REFUNDED**.
- Refundable balance should become **0**.

Actual Result

The refund was processed successfully. The order status changed to **REFUNDED** and the refundable balance became **0**.

Test 2 – Verify Partial Refund Amount and Tax Distribution

Description

This test verifies that a partial refund correctly calculates the refund amount and tax.

Steps Performed

1. Created a refund test order.
2. Refunded one item.
3. Verified the refund amount.
4. Verified the line amount.
5. Verified the tax amount.

6. Verified that all tax shares together equal the total tax.

Expected Result

- Refund Amount = **34965 paise**
- Line Amount = **33300 paise**
- Tax = **1665 paise**
- Total Tax Share = **4995 paise**

Actual Result

The refund amount and tax calculations were correct.

Test 3 – Verify Refund Eligibility Rules

Description

This test verifies that the refund eligibility API returns the correct verdict for different scenarios.

Scenario 1 – Approved Refund

Steps

- Created a refund order.
- Requested refund eligibility for one item.

Expected Result

Verdict should be **APPROVED**.

Actual Result

API returned **APPROVED**.

Scenario 2 – Over Refund

Steps

- Created a refund order.
- Requested refund eligibility for four items although only three were purchased.

Expected Result

Verdict should be **OVER_REFUND**.

Actual Result

API returned **OVER_REFUND**

Test 4 – Verify Over Refund is Rejected

Description

This test verifies that the system does not allow refunding more items than were originally purchased.

Steps Performed

1. Created a refund order.
2. Requested refund for four items.
3. Verified the API response.

Expected Result

- Response Status = **422**
- Verdict = **OVER_REFUND**

Actual Result

The application rejected the refund request with status **422** and returned the verdict **OVER_REFUND**.

Test 5 – Verify Idempotent Refund

Description

This test verifies that sending the same refund request multiple times with the same **Idempotency-Key** does not create duplicate refunds.

Steps Performed

1. Created a refund order.
2. Generated one Idempotency-Key.
3. Sent the first refund request.
4. Sent the same refund request again using the same key.
5. Retrieved the updated order details.
6. Verified the refund count.

Expected Result

- First request returns **201**.
- Second request returns **200**.
- Refund Count remains **1**.

Actual Result

The application replayed the previous response and did not create another refund. The refund count remained **1**.

Failures:

In the test 5 the same idempotency key creates only one refund the count of the refund should be 1 for the failure changed to refundcount 2 so the test is failing.

```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests> npx playwright test refund.spec.ts --reporter=html
Error: expect(received).toBe(expected) // Object.is equality

Expected: 2
Received: 1

261 |     const finalOrder = await orderDetails.json();
262 |     //the expected refundCount is 1 changed 2
> 263 |     expect(finalOrder.refundCount).toBe(2);
    |                                     ^
264 |
265 |     evidence.cartResponse = await finalOrder;
266 |   });
    at C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests\refund.spec.ts:263:34
```

> ✓ Expect "toBe" — refund.spec.ts:250	0ms
> ✓ GET "/api/refund-lab/orders/7203" — refund.spec.ts:252	3ms
> ✓ Expect "toBe" — refund.spec.ts:259	1ms
> ✗ Expect "toBe" — refund.spec.ts:263	3ms
> ✓ After Hooks	43ms
✓ Worker Cleanup	3ms

Attachments

>  cart-response.json

>  diagnosis.txt

 logs.txt

 [error-context](#)

> Executed in Worker #0

From the report I got to know the error and I changed the expected the refund count to the 1 so it passed.

```
PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests> npx playwright test refund.spec.ts --reporter=html

Running 6 tests using 1 worker
 6 passed (1.7s)

To
  Run npx playwright show-report (ctrl + click)
  npx playwright show-report

PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests>
```

> ✓ Expect "toBe" — refund.spec.ts:228	1ms
> ✓ POST "/api/refunds" — refund.spec.ts:230	3ms
> ✓ Expect "toBe" — refund.spec.ts:250	0ms
> ✓ GET "/api/refund-lab/orders/7209" — refund.spec.ts:252	2ms
> ✓ Expect "toBe" — refund.spec.ts:259	0ms
> ✓ Expect "toBe" — refund.spec.ts:264	0ms
> ✓ After Hooks	10ms

Attachments

cart-response.json

```
{
  "id": 7209,
  "orderNumber": "RET-7209",
  "ownerKey": "user-1",
  "placedOn": "2026-06-29",
  "daysAgo": 5,
  "returnWindowDays": 30,
  "paymentMethod": "Original card",
  "lines": [
    {
      "sku": "TEE",
      "name": "Training Tee",
      "unitPrice": 33.00
    }
  ]
}
```

Flakiness Verification

The test was executed 50 consecutive times using Playwright's `--repeat-each=50` command to verify stability.

Command Used

`npx playwright test testing-resiliancy.spec.ts --repeat-each=50`

Result

- Total Executions: 30
- Passed: 25
- Failed: 05

```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT
PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests> npx playwright test testing-resiliency.spec.ts --repeat-each=30
12 |
    at openCleanPost (C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests\testing-resiliency.spec.ts:9:16)
    at C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests\testing-resiliency.spec.ts:183:5

Error Context: ..\test-results\testing-resiliency-Testing-46a4f-for-offline-to-sync-journey-repeat23\error-context.md

5 failed
testing-resiliency.spec.ts:180:8 › Testing Resiliency › creates evidence for offline to sync journey
testing-resiliency.spec.ts:180:8 › Testing Resiliency › creates evidence for offline to sync journey
testing-resiliency.spec.ts:180:8 › Testing Resiliency › creates evidence for offline to sync journey
testing-resiliency.spec.ts:180:8 › Testing Resiliency › creates evidence for offline to sync journey
testing-resiliency.spec.ts:180:8 › Testing Resiliency › creates evidence for offline to sync journey
25 passed (1.7m)
1 error was not a part of any test, see above for details
PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests> |
```

Issue

Some test cases failed intermittently because of timing/network delays.

Root Cause

The failure was temporary and not caused by an application defect.

Fix

Configured Playwright retries:

retries: 2

Enabled:

trace: 'on-first-retry'

screenshot: 'only-on-failure'

video: 'retain-on-failure'

Result

- Failed tests were retried automatically.
- Flaky tests passed on retry.

Results:

Ran the all testcases by giving command **npx playwright test**.

```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS PLAYWRIGHT +
PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests> npx playwright test
{"2026-07-04T11:41:56.792Z","workerIndex":0}
  ✓ 9 ...gging.spec.ts:90:9 › Diagnostics and Logging › redacts sensitive fields in structured logs (399ms)
  ✓ 2 ...8:9 › Testing Resiliency › rolls back an online optimistic sale when the sync request fails (2.3s)
  ✓ 10 ...esiliency.spec.ts:63:9 › Testing Resiliency › queued sale retries with same idempotency key (1.9s)
  ✓ 11 testing-resiliency.spec.ts:111:7 › Testing Resiliency › detects and displays offline banner (897ms)
  ✓ 12 testing-resiliency.spec.ts:128:8 › Testing Resiliency › queues a sale while offline (1.2s)
  ✓ 13 testing-resiliency.spec.ts:143:8 › Testing Resiliency › syncs queued sale on reconnect (1.1s)
  ✓ 14 ...siliency.spec.ts:180:7 › Testing Resiliency › creates evidence for offline to sync journey (989ms)

14 passed (12.9s)
❖ PS C:\Users\304556\Desktop\testing\week5-gate5\SDET\tests>
Ln 12, Col 2 Spaces: 4 UTF-8 CRLF { } TypeScript
```